

Sri Lanka Institute of Information Technology



BSc (Hons) in Information Technology Specializing in Data Science

IT3021 – Data Warehousing and Business Intelligence

Year 3 Semester 1-2026

Assignment 1

IT23858398

DILRANGA M.M.P.M

Table Of Contents

1	STEP 1: DATA SET SELECTION.....	4
1.1	INTRODUCTION.....	4
1.2	JUSTIFICATION FOR SELECTION.....	4
1.3	DATASET OVERVIEW.....	4
1.4	CONCEPTUAL DATA MODEL.....	5
1.5	FIGURE 1 SHOWS A SAMPLE OF THE ORIGINAL DATASET.....	5
1.6	ER-DIAGRAM.....	6
2	STEP 2:PREPARATION OF DATA SOURCES	7
2.1	DATA SPLITTING STRATEGY.....	7
2.1.1	CSV File (Accident Core Data).....	7
2.1.2	Excel File (Road Conditions).....	7
2.1.3	SQL Server Table (Casualty & Vehicle).....	7
2.2	FIGURE 2 ILLUSTRATES HOW DATA WAS SPLIT INTO MULTIPLE SOURCES.....	8
2.3	SOURCE DATABASE IMPLEMENTATION.....	8
2.4	DATA QUALITY CONSIDERATIONS.....	9
3	STEP 3: PREPARATION OF DATA SOURCES	10
3.1	OVERVIEW.....	10
3.2	ARCHITECTURE COMPONENTS.....	10
3.2.1	Source Layer.....	10
3.2.2	Staging Layer.....	10
3.2.3	Data Warehouse Layer.....	10
3.2.4	ETL Layer.....	11
3.2.5	Reporting Layer.....	11
3.2.6	Figure 3: Solution Architecture Diagram.....	11
4	STEP 4:DATA WAREHOUSE DESIGN & DEVELOPMENT.....	12
4.1	SCHEMA DESIGN.....	12
4.2	FACT TABLE – FACTACCIDENT.....	12
4.2.1	Measures:.....	12
4.2.2	Foreign Keys:.....	12
4.3	DIMENSION TABLES.....	12
4.4	SLOWLY CHANGING DIMENSION IMPLEMENTATION.....	12
4.5	IMPLEMENTATION.....	13
4.6	FIGURE 4: STAR SCHEMA DIAGRAM.....	13
5	STEP 5:ETL DEVELOPMENT.....	14
5.1	OVERVIEW.....	14
5.2	CONNECTION MANAGERS.....	14
5.3	PACKAGE 1 – STAGING LOAD.....	14
5.4	PACKAGE 2 – DATA WAREHOUSE LOAD.....	16
5.4.1	Data Flow – Transform and Load Location Dimension.....	16
5.4.2	Data Flow – Transform and Load Road Condition Dimension.....	17
5.4.3	Data Flow – Transform and Load Casualty Dimension.....	19
5.4.4	Data Flow – Transform and Load Accident Fact.....	20

5.4.5	<i>Validation of Package 2 Load</i>	22
5.5	ERROR HANDLING AND DATA QUALITY CHECKS	23
5.6	ETL EXECUTION ORDER	24
6	STEP 6: ACCUMULATING FACT TABLE	24
6.1	OVERVIEW	24
6.2	FACT TABLE EXTENSION	24
6.3	IMPLEMENTATION LOGIC	25
6.4	PACKAGE 3 - ETL PACKAGE FOR FACT UPDATE	25
6.5	VALIDATION	28
7	STEP 7: CONCLUSION	29
8	ASSUMPTIONS	29

1 Step 1: Data Set Selection

1.1 Introduction

In this project, a real-world dataset titled “**UK Road Accident Severity and Risk Analysis – 2009**” was selected. This dataset contains detailed records of road traffic accidents that occurred in the United Kingdom during the year 2009. It includes multiple attributes describing accident characteristics, environmental conditions, and casualty details.

1.2 Justification for Selection

This dataset was chosen based on the following criteria:

- It represents a **true OLTP-style dataset**, not pre-modeled for OLAP.
- Contains **over 3000 records**, providing sufficient data volume.
- Covers a **full year**, enabling time-based analysis.
- Includes multiple related entities such as:
 - Accident details
 - Road conditions
 - Casualties
- Supports creation of:
 - Multiple dimensions
 - Fact table
 - Hierarchies and aggregations

Therefore, this dataset is suitable for demonstrating data warehousing concepts and ETL processes.

1.3 Dataset Overview

The dataset contains the following types of information:

- **Accident Information**
 - Reference Number
 - Date and Time
 - Number of Vehicles
- **Location Information**
 - Easting and Northing
 - Road Class
- **Environmental Conditions**
 - Weather Conditions
 - Lighting Conditions
 - Road Surface

- **Casualty Information**
 - Casualty Class
 - Casualty Severity
 - Age and Gender
 - Vehicle Type

1.4 Conceptual Data Model

From the dataset, the following conceptual entities were identified:

- Accident
- Location
- Road Condition
- Casualty

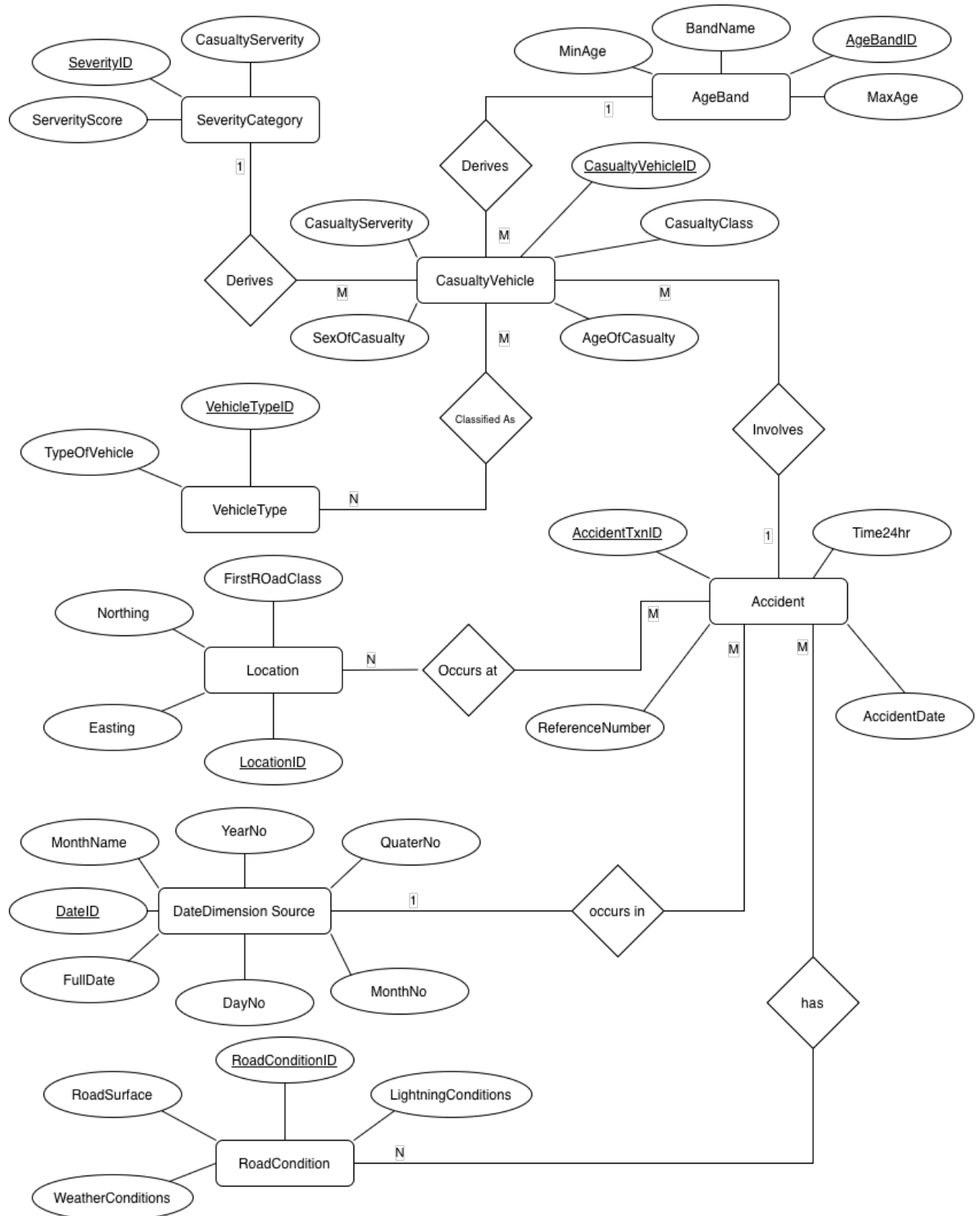
Relationships:

- One accident occurs at one location
- One accident has one road condition
- One accident can involve multiple casualties

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	Reference N	Easting	Northing	Number of Ve	Accident Dat Time (24hr)	1st Road Cla	Road Surface	Lighting Con	Weather Cor	Casualty Cla	Casualty Sev	Sex of Casua	Age of Casua	Type of Vehicle							
1	3309	429093	436258	1	1-Jan-09	55	Unclassified	Dry	Darkness: str	Fine without	Pedestrian	Slight	Male	44	Car						
3	2609	434723	435534	1	2-Jan-09	2335	Unclassified	Dry	Darkness: str	Fine without	Driver	Serious	Female	23	Car						
4	2809	441173	433047	1	2-Jan-09	1645	Unclassified	Dry	Darkness: str	Fine without	Pedestrian	Slight	Female	12	Car						
5	3809	428457	431364	1	2-Jan-09	1723	A	Dry	Darkness: str	Fine without	Pedestrian	Slight	Male	15	Car						
6	3909	425928	435480	2	2-Jan-09	1350	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Female	34	Car						
7	3909	425928	435480	2	2-Jan-09	1350	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	25	Car						
8	19809	425642	434240	2	2-Jan-09	2300	A	Dry	Darkness: str	Fine without	Driver	Slight	Male	57	Car						
9	3709	429610	439798	1	3-Jan-09	222	Unclassified	Dry	Darkness: str	Fine without	Passenger	Slight	Male	17	Car						
10	4309	426535	433642	2	3-Jan-09	325	Unclassified	Wet / Damp	Darkness: str	Fine without	Passenger	Slight	Female	23	Car						
11	9009	437658	445061	1	3-Jan-09	1645	A	Wet / Damp	Darkness: no	Fine without	Passenger	Slight	Male	20	Car						
12	9009	437658	445061	1	3-Jan-09	1645	A	Wet / Damp	Darkness: no	Fine without	Passenger	Slight	Male	23	Car						
13	66209	431508	435120	2	3-Jan-09	1745	Unclassified	Dry	Darkness: str	Fine without	Driver	Slight	Female	18	Car						
14	66209	431508	435120	2	3-Jan-09	1745	Unclassified	Dry	Darkness: str	Fine without	Passenger	Slight	Female	47	Car						
15	66209	431508	435120	2	3-Jan-09	1745	Unclassified	Dry	Darkness: str	Fine without	Passenger	Slight	Male	11	Car						
16	66209	431508	435120	2	3-Jan-09	1745	Unclassified	Dry	Darkness: str	Fine without	Passenger	Slight	Female	15	Car						
17	66209	431508	435120	2	3-Jan-09	1745	Unclassified	Dry	Darkness: str	Fine without	Passenger	Slight	Female	42	Car						
18	2909	443402	438684	2	4-Jan-09	1605	A(M)	Dry	Darkness: str	Fine without	Driver	Slight	Male	83	Car						
19	2909	443402	438684	2	4-Jan-09	1605	A(M)	Dry	Darkness: str	Fine without	Passenger	Slight	Female	65	Car						
20	4209	428099	426648	1	4-Jan-09	21	A	Wet / Damp	Darkness: str	Fine without	Driver	Slight	Female	24	Car						
21	8409	431482	435007	2	4-Jan-09	1430	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	28	Car						
22	8409	431482	435007	2	4-Jan-09	1430	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	29	Car						
23	8409	431482	435007	2	4-Jan-09	1430	Unclassified	Dry	Daylight: str	Fine without	Passenger	Slight	Male	26	Car						
24	3109	427890	430474	1	5-Jan-09	705	A	Wet / Damp	Darkness: str	Fine without	Driver	Slight	Male	42	Car						
25	3109	427890	430474	1	5-Jan-09	705	A	Wet / Damp	Darkness: str	Fine without	Pedestrian	Fatal	Male	18	Car						
26	8509	427304	433623	3	5-Jan-09	1430	A	Dry	Daylight: str	Fine without	Driver	Slight	Male	34	Car						
27	8509	427304	433623	3	5-Jan-09	1430	A	Dry	Daylight: str	Fine without	Driver	Slight	Female	38	Car						
28	11009	432430	433799	1	5-Jan-09	1250	Unclassified	Dry	Daylight: str	Fine without	Passenger	Slight	Male	40	Bus or coach (17 or more passenger seats)						
29	22509	431463	432466	1	5-Jan-09	2330	A	Dry	Darkness: str	Fine without	Driver	Slight	Male	23	Car						
30	8609	432533	434156	1	6-Jan-09	1805	Unclassified	Dry	Darkness: str	Fine without	Pedestrian	Slight	Female	4	Car						
31	9109	421617	434392	2	6-Jan-09	1800	B	Dry	Darkness: str	Fine without	Driver	Slight	Male	78	Car						
32	9109	421617	434392	2	6-Jan-09	1800	B	Dry	Darkness: str	Fine without	Driver	Slight	Male	51	Car						
33	11609	433601	429802	1	6-Jan-09	1725	Motorway	Dry	Darkness: str	Fine without	Pedestrian	Slight	Male	52	Car						
34	12909	424112	434008	1	6-Jan-09	1308	A	Dry	Daylight: str	Fine without	Passenger	Slight	Male	66	Bus or coach (17 or more passenger seats)						
35	13109	425186	437731	2	6-Jan-09	1040	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	30	Taxi/Private hire car						
36	13109	425186	437731	2	6-Jan-09	1040	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	66	Car						
37	19109	421016	441155	1	6-Jan-09	1120	Unclassified	Dry	Daylight: str	Fine without	Pedestrian	Slight	Male	88	Car						
38	21809	429870	430441	3	6-Jan-09	2055	Unclassified	Wet / Damp	Darkness: str	Fine without	Driver	Slight	Male	40	Car						
39	22309	423629	437888	3	6-Jan-09	1109	Unclassified	Dry	Daylight: str	Fine without	Driver	Slight	Male	49	M/cycle 50cc and under						

1.5 Figure 1 shows a sample of the original dataset

1.6 ER-Diagram



2 Step 2:Preparation of Data Sources

2.1 Data Splitting Strategy

The original dataset was provided as a **single CSV file**, which does not reflect real world data environments. Therefore, the dataset was split into multiple sources to simulate a realistic ETL scenario.

The splitting was done based on logical grouping of attributes.

Created Data Sources

Three different data sources were created:

2.1.1 CSV File (Accident Core Data)

Contains:

- Reference Number
- Date and Time
- Location coordinates
- Number of vehicles

2.1.2 Excel File (Road Conditions)

Contains:

- Road Surface
- Lighting Conditions
- Weather Conditions

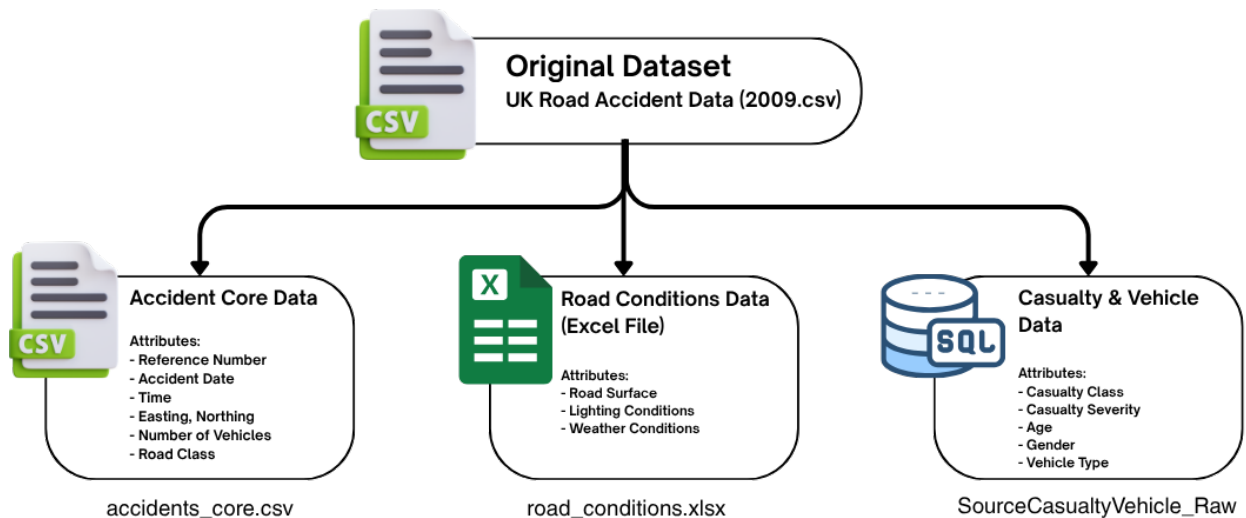
2.1.3 SQL Server Table (Casualty & Vehicle)

Contains:

- Casualty Class
- Casualty Severity
- Age and Gender
- Vehicle Type

This ensures:

- At least two different source types (CSV, Excel, Database)
- Realistic ETL complexity



2.2 Figure 2 illustrates how data was split into multiple sources

2.3 Source Database Implementation

To support the ETL workflow and simulate a realistic operational environment, a dedicated SQL Server source database named `UKRoadAccident_SourceDB` was created. This database was used to store the source data that was prepared for extraction into the staging layer.

Although the original dataset was available as a single flat file, the assignment required the use of multiple source types. Therefore, part of the dataset was loaded into SQL Server so that the ETL process could demonstrate extraction from both file-based and database-based systems.

Within `UKRoadAccident_SourceDB`, the casualty and vehicle-related portion of the dataset was first loaded into a raw source table named `SourceCasualtyVehicle_Raw`. This raw table was designed to accept the imported file with minimal structural restrictions, allowing data to be captured even when inconsistencies such as encoding issues, varying text lengths, or mixed field formats were present. Storing the imported records in a raw structure made it possible to perform controlled cleaning and datatype standardization before the data was used in later ETL stages.

After the raw import, the data was cleansed and inserted into a structured source table named `SourceCasualtyVehicle`. This cleaned table contained the finalized source data used by SSIS during extraction to the staging layer. The implementation separated raw ingestion from cleaned source storage, which is a common best practice in data integration projects because it improves traceability, reduces data loss during import, and allows source-level validation before staging.

An artificial transaction identifier named `AccidentTxnID` was introduced during source preparation. This field was necessary because the original accident reference number was not unique across all rows in the dataset. Since multiple records could exist for the same accident due to different casualties or vehicles, `AccidentTxnID` was used as a unique row level identifier. This key was then consistently maintained across all split sources so that records from CSV, Excel, and SQL Server could be joined accurately during ETL processing.

The final SQL source table stored the following casualty and vehicle-related attributes:

- `AccidentTxnID`
- `ReferenceNumber`
- `CasualtyClass`
- `CasualtySeverity`
- `SexOfCasualty`
- `AgeOfCasualty`
- `TypeOfVehicle`

This source database played an important role in the overall architecture because it enabled the project to demonstrate heterogeneous source integration, data cleansing, and controlled source-to-staging extraction using SQL Server and SSIS.

2.4 Data Quality Considerations

During the preparation of data sources, several data quality issues were identified in the original dataset. These issues needed to be addressed to ensure smooth ETL processing and accurate data warehouse results.

One major issue was the presence of duplicate reference numbers, as a single accident could have multiple related records (e.g., multiple casualties). To handle this, a new unique identifier called **`AccidentTxnID`** was introduced to uniquely identify each record. Additionally, some columns contained inconsistent data types, such as numeric values stored as text. These were handled during the ETL process using data conversion transformations.

There were also cases of missing or null values in certain attributes such as age or road conditions. These values were retained as NULL and handled appropriately during transformation.

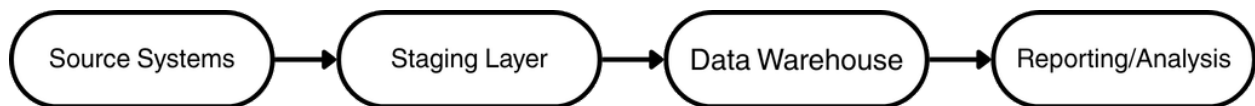
Another issue encountered was encoding and formatting inconsistencies when importing CSV files, which were resolved by standardizing file formats and column names. By identifying and addressing these issues early, the data was successfully prepared for reliable staging and data warehouse loading.

3 Step 3: Preparation of Data Sources

3.1 Overview

The solution was designed using a layered data warehousing architecture to ensure proper separation of concerns and efficient data processing. The architecture follows a standard flow from data sources to the final data warehouse.

The overall flow is:



3.2 Architecture Components

3.2.1 Source Layer

The source layer consists of multiple data sources created from the original dataset. These include CSV files, an Excel file, and a SQL Server table. This simulates a real-world scenario where data originates from different systems and formats.

3.2.2 Staging Layer

A staging database named **UKRoadAccident_Staging** was created to temporarily store extracted data.

The staging layer is responsible for:

- Holding raw data from all sources
- Standardizing data types
- Performing initial cleaning
- Preparing data for transformation

No business logic is applied at this stage .it is mainly used for preprocessing.

3.2.3 Data Warehouse Layer

The data warehouse was implemented in a database named **UKRoadAccident_DW**.

This layer contains:

- Dimension tables
- Fact table

The data is structured in a dimensional model to support efficient querying and analysis.

3.2.4 ETL Layer

The ETL process was implemented using **SQL Server Integration Services (SSIS)**.

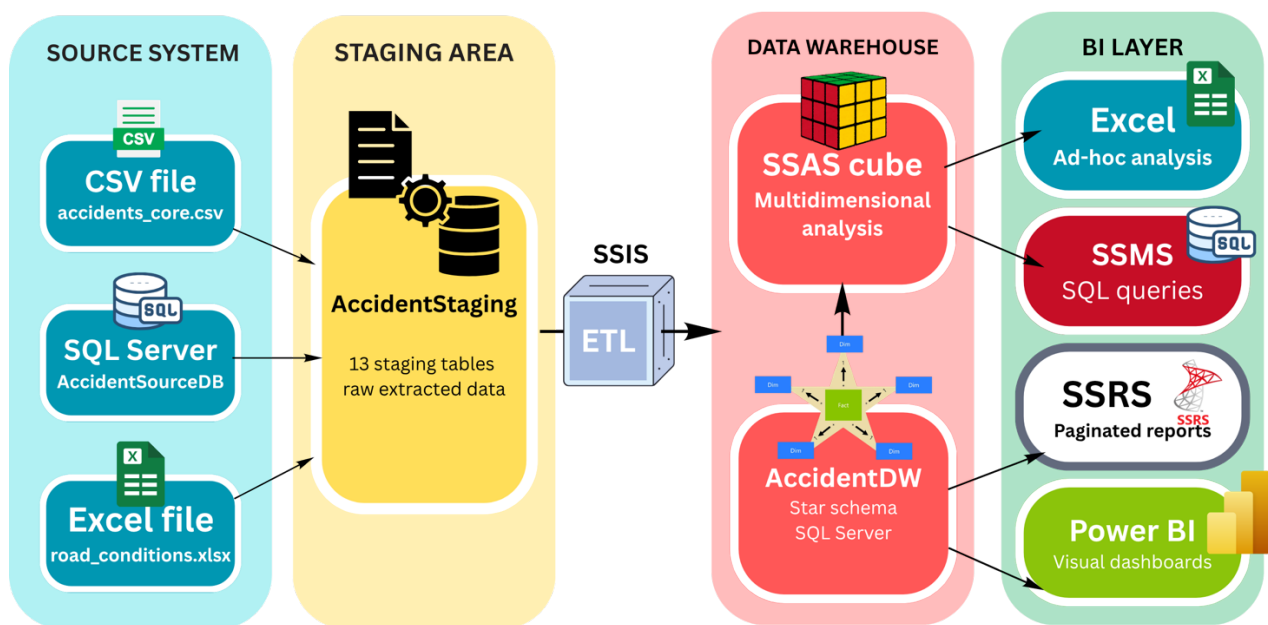
The ETL layer performs:

- Data extraction from multiple sources
- Data transformation (cleaning, enrichment, conversions)
- Data loading into staging and warehouse

3.2.5 Reporting Layer

The final data warehouse is designed to support analytical tools such as SSAS or Power BI. This layer enables reporting, aggregation, and decision making.

3.2.6 Figure 3: Solution Architecture Diagram



4 Step 4:Data Warehouse Design & Development

4.1 Schema Design

A **star schema** was used to design the data warehouse due to its simplicity and efficiency for analytical queries. The schema consists of one central fact table connected to multiple dimension tables.

4.2 Fact Table – FactAccident

The **FactAccident** table stores measurable data related to each accident event.

4.2.1 Measures:

- NumberOfVehicles
- SeverityScore
- txn_process_time_hours

4.2.2 Foreign Keys:

- DateKey
- LocationKey
- RoadConditionKey
- CasualtyKey

Each record in the fact table represents a single accident transaction.

4.3 Dimension Tables

DimDate:- Stores date-related information such as year, month, and day, enabling time-based analysis.

DimLocation:- Stores location details including Easting, Northing, and road class.

DimRoadCondition:- Stores environmental conditions such as weather, lighting, and road surface.

DimCasualty:- Stores casualty-related information including class, severity, age group, and vehicle type.

4.4 Slowly Changing Dimension Implementation

The **DimCasualty** table includes derived information such as **AgeBand**, which categorizes age into groups (e.g., Child, Adult, Senior). This allows better analytical grouping.

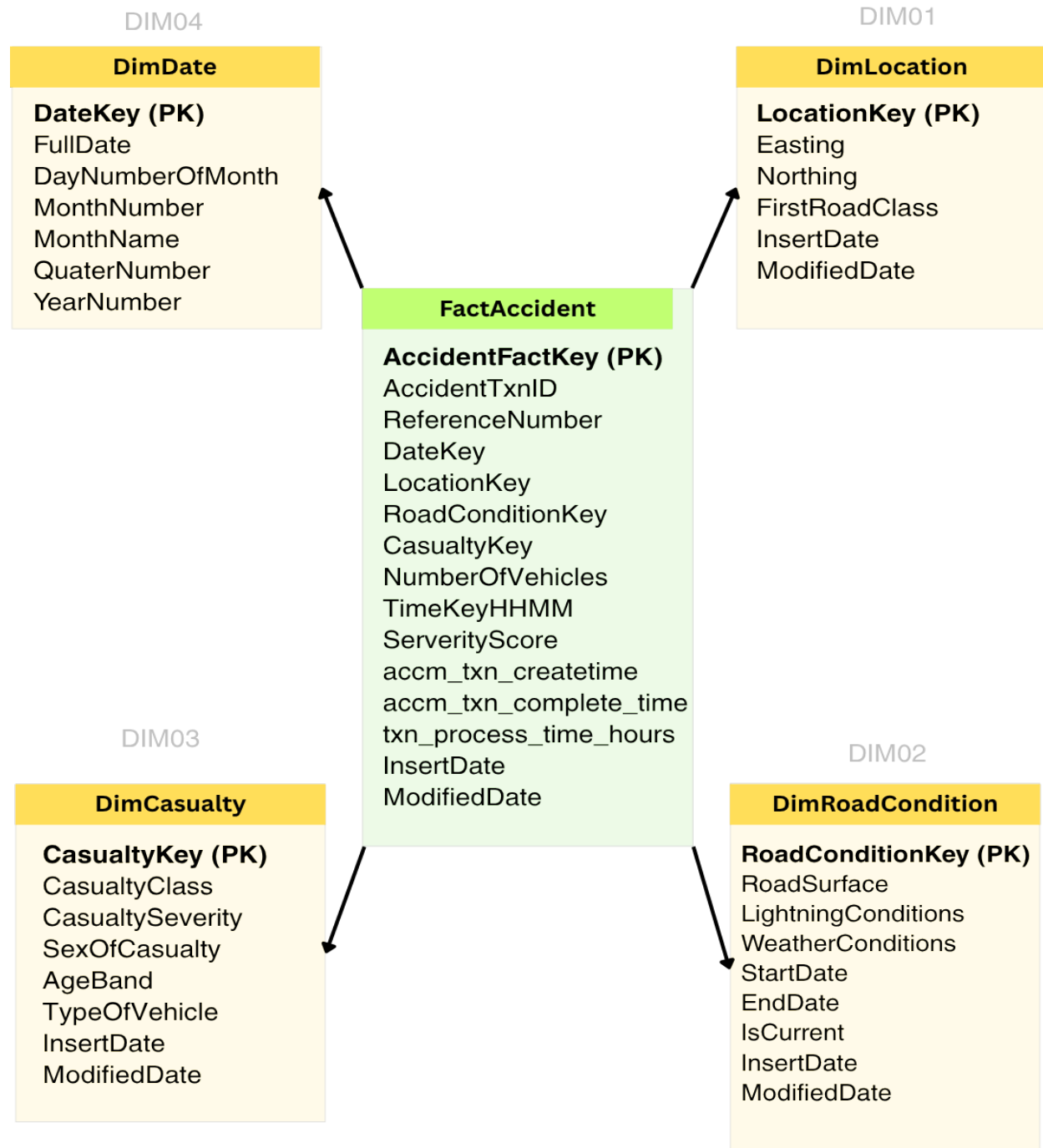
4.5 Implementation

All tables were implemented in SQL Server with appropriate:

- Primary keys
- Foreign key relationships
- Data types

The design ensures data consistency and efficient querying.

4.6 Figure 4: Star Schema Diagram



5 Step 5:ETL Development

5.1 Overview

The ETL process was developed using SSIS to extract data from multiple sources, transform it into a consistent format, and load it into the data warehouse.

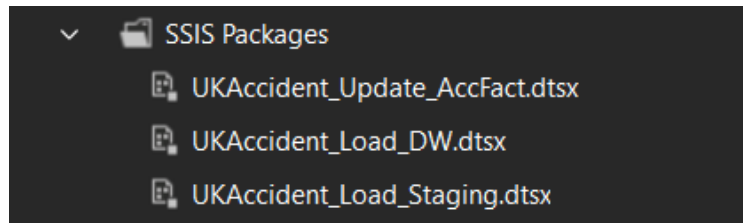


Figure 5 : SSIS Package Modules as Seen in Solution Explorer

5.2 Connection Managers

All ETL packages utilize multiple connection types to handle data from both databases and files. The following connections were configured

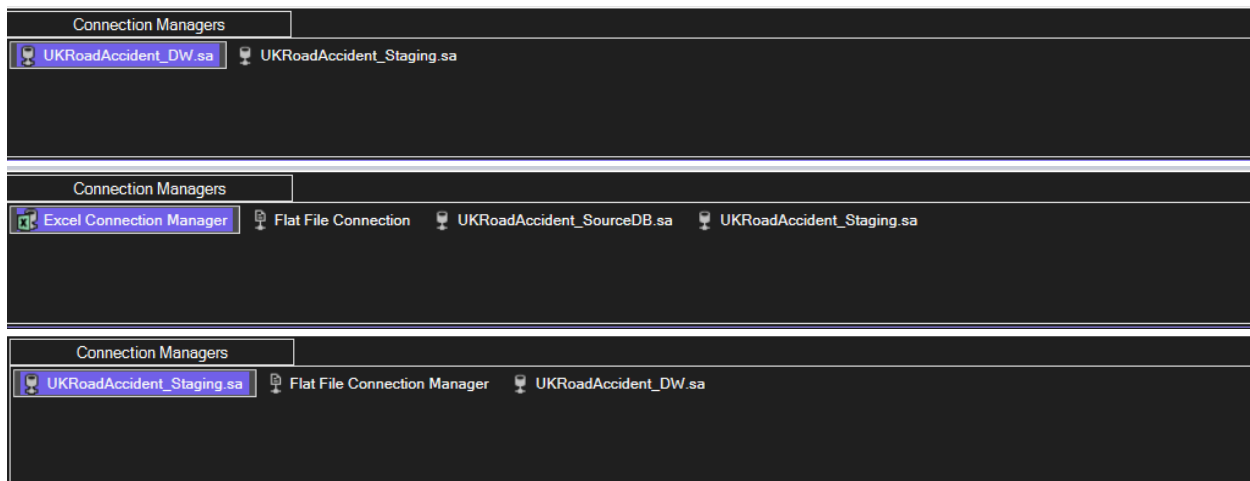


Figure 6: Connection Manager panels showing database and flat file sources.

5.3 Package 1 – Staging Load

The first ETL stage loads data into the staging database.

Steps involved:

- Extract data from CSV, Excel, and SQL sources
- Perform basic data type conversions
- Load data into staging tables

This step ensures all data is centralized before further processing.

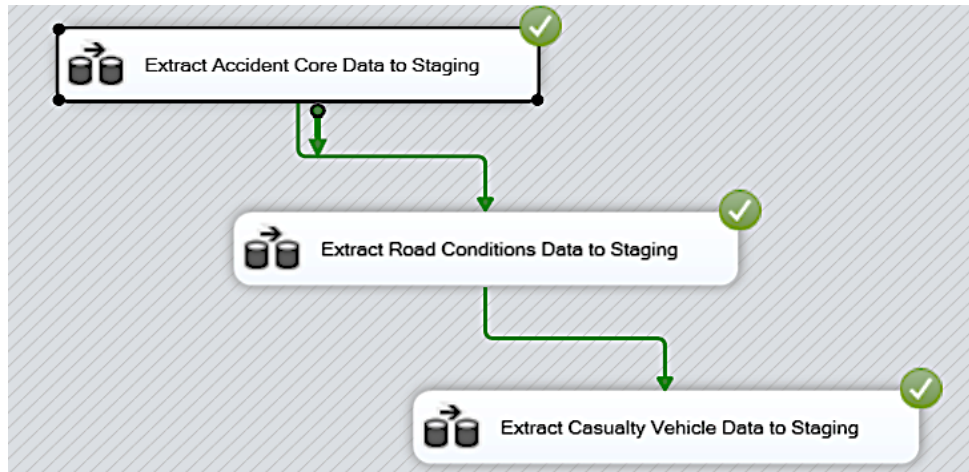


Figure 7: Control Flow in UKAccident_Load_Staging.dtsx showing staging tasks.

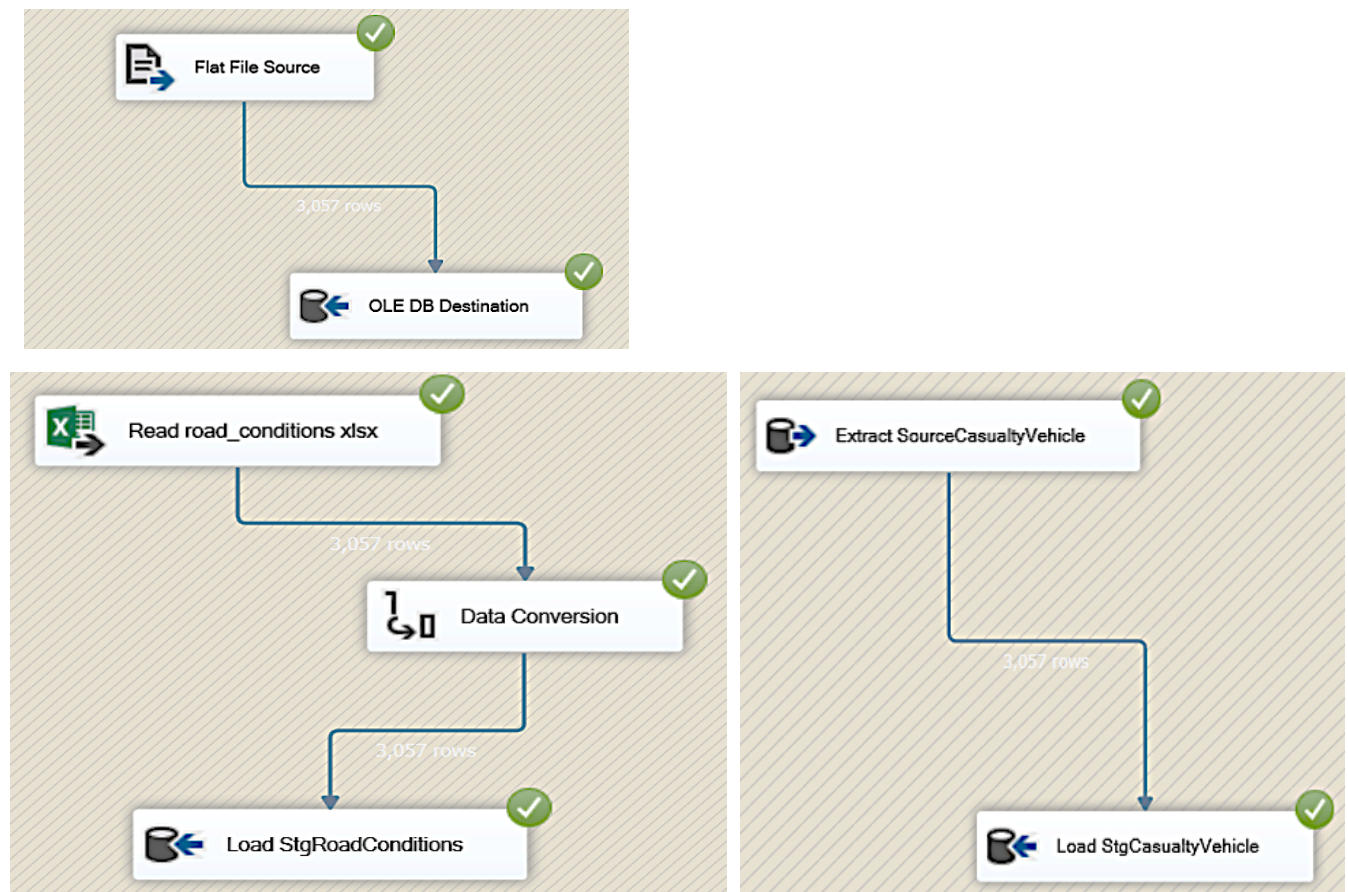


Figure 8: Data Flows from source to staging tables

5.4 Package 2 – Data Warehouse Load

The second ETL stage loads data from staging into the data warehouse. This package handles the transformation and loading of clean data into the dimensional and fact tables of the data warehouse. The tasks are executed in the following order:

1. Transform and Load Location Dimension
2. Transform and Load Road Condition Dimension
3. Transform and Load Casualty Dimension
4. Transform and Load Accident Fact

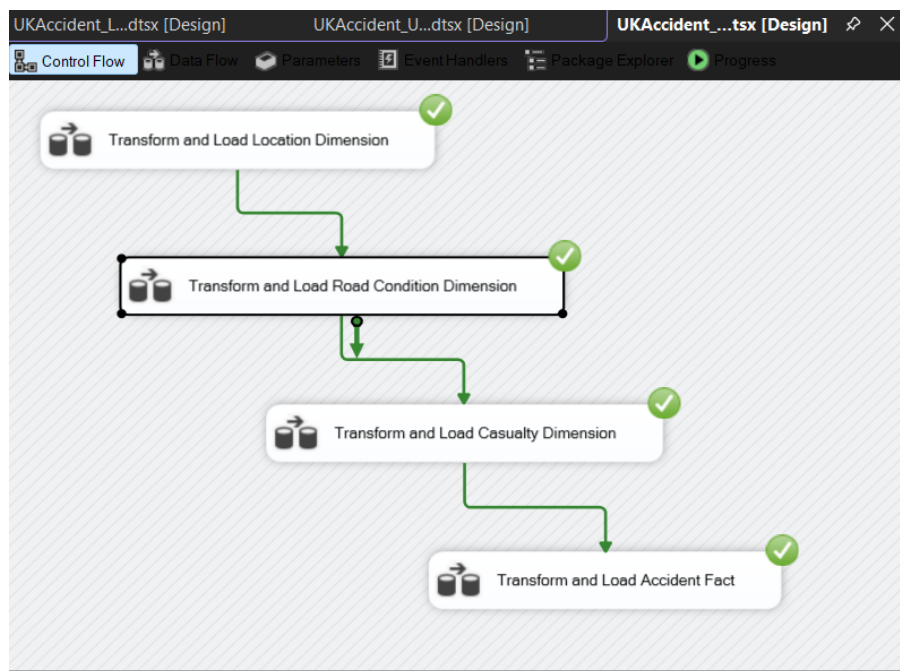


Figure 9: Control Flow in UKAccident_Load_DW.dtsx showing dimensional and fact loading sequence.

5.4.1 Data Flow – Transform and Load Location Dimension

This data flow loads location-related attributes from the staging layer into the DimLocation table in the data warehouse.

The source of this flow is the StgAccidentCore staging table. Only the location-related attributes were selected:

- Easting
- Northing
- FirstRoadClass

A SELECT DISTINCT query was used to reduce repeated values before loading. A Sort transformation was then used to remove any remaining duplicate rows and prepare a clean set of location records.

The cleaned records were loaded into the DimLocation table using an OLE DB Destination. Since LocationKey is an identity-based surrogate key, it was generated automatically by SQL Server during insertion.

This dimension was created to support analysis of accidents by geographical coordinates and road classification.

Main components used

- OLE DB Source
- Sort
- OLE DB Destination

To populate the location dimension with unique location records derived from the source accident data.

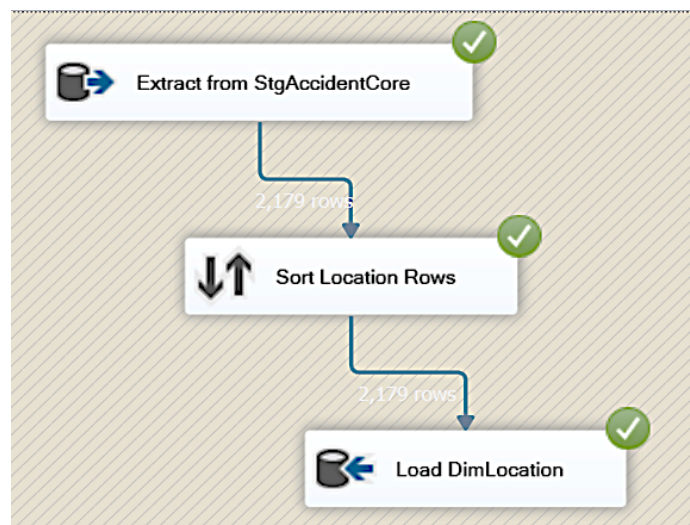


Figure 10: Data Flow Transform and Load Location Dimension

5.4.2 Data Flow – Transform and Load Road Condition Dimension

This data flow loads road and environmental condition data from the staging layer into the DimRoadCondition table.

The source for this transformation is the StgRoadConditions table, which contains:

- RoadSurface
- LightingConditions
- WeatherConditions

A SELECT DISTINCT query was used to identify unique combinations of road condition attributes. A Sort transformation was then applied to remove any duplicates.

After that, a Derived Column transformation was used to add Slowly Changing Dimension-related fields:

- StartDate
- EndDate
- IsCurrent

These fields prepare the dimension for SCD-style tracking. Although the load was implemented as a full load during development, the structure supports historical tracking in the future.

The final records were inserted into DimRoadCondition using an OLE DB Destination.

Main components used

- OLE DB Source
- Sort
- Derived Column
- OLE DB Destination

To create a dimension containing environmental and road-related descriptive information for accident analysis.

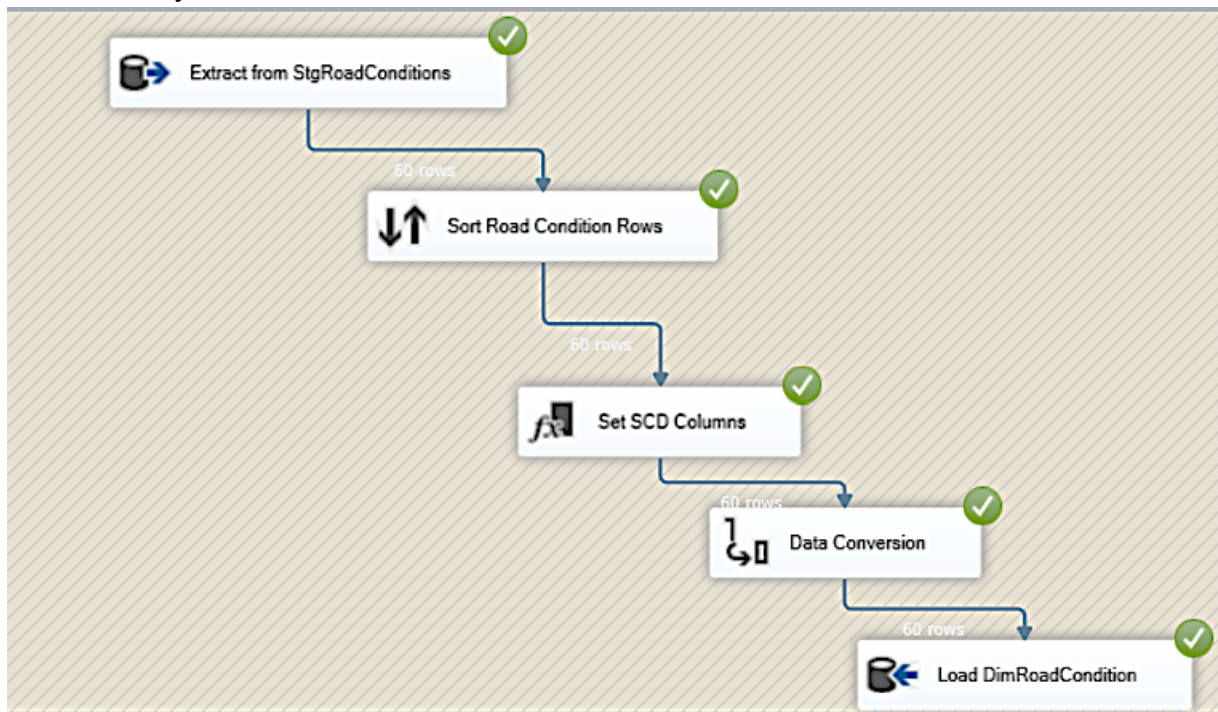


Figure 11: Data Flow Transform and Load Road Condition Dimension

5.4.3 Data Flow – Transform and Load Casualty Dimension

This data flow loads casualty and vehicle-related attributes into the DimCasualty table.

The source is the StgCasualtyVehicle staging table, which contains the following fields:

- CasualtyClass
- CasualtySeverity
- SexOfCasualty
- AgeOfCasualty
- TypeOfVehicle

A Derived Column transformation was used to generate a new analytical attribute called AgeBand. This grouped casualty ages into broader categories, making the data more suitable for reporting and aggregation.

Example groupings include:

- 0–17
- 18–29
- 30–44
- 45–59
- 60+
- Unknown

After deriving AgeBand, a Sort transformation was applied to remove duplicate casualty records. The unique rows were then loaded into DimCasualty using an OLE DB Destination.

This dimension helps analyze accident severity and casualties based on age groups, gender, class, and vehicle type.

Main components used

- OLE DB Source
 - Derived Column
 - Sort
 - OLE DB Destination
- Purpose

To populate the casualty dimension with descriptive casualty and vehicle attributes enriched with derived age bands.

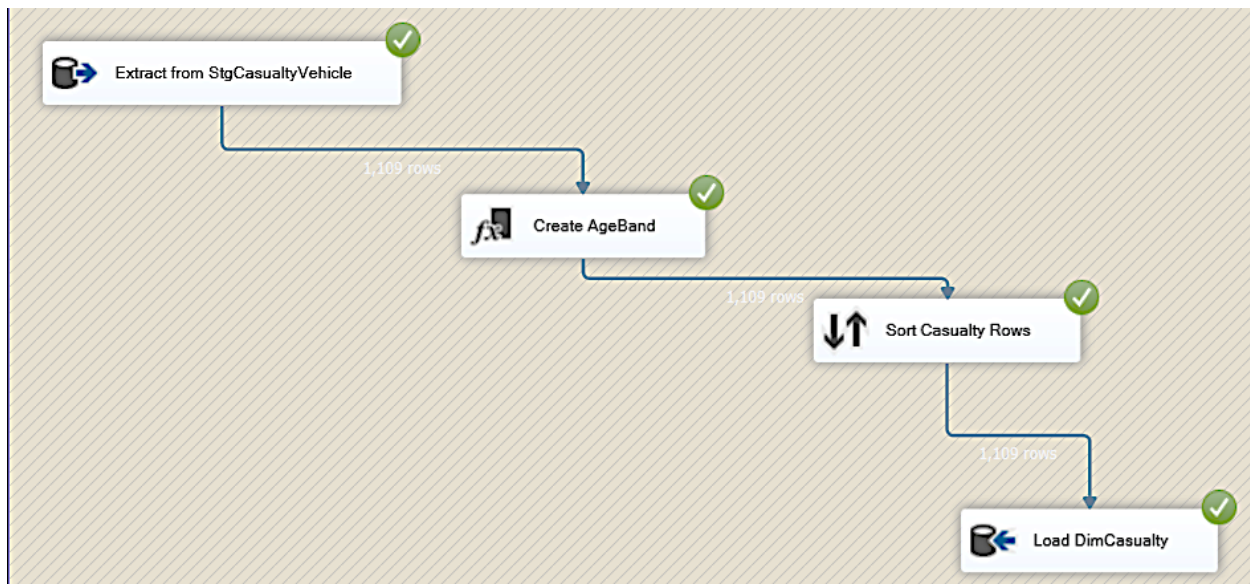


Figure 12: Data Flow Transform and Load Casualty Dimension

5.4.4 Data Flow – Transform and Load Accident Fact

This is the main fact-loading data flow in the package. It integrates data from the three staging tables and loads the final result into the FactAccident table.

The data flow begins with a joined source query combining:

- StgAccidentCore
- StgRoadConditions
- StgCasualtyVehicle

The join was performed using AccidentTxnID, which was introduced as a unique row-level identifier during source preparation. This allowed the three heterogeneous sources to be recombined accurately during ETL.

The following transformations were then applied:

1. Derived Column

Used to create additional analytical fields such as:

- AgeBand
- TimeKeyHHMM
- SeverityScore
- accm_txn_create_time

SeverityScore was derived from CasualtySeverity to create a numeric value that can be used in analysis.

2. Data Conversion

Used to convert string and non-Unicode values into compatible datatypes required for lookup operations and loading. This was especially important for matching source columns with dimension table columns.

3. Lookup Transformations

Several lookup transformations were used to retrieve surrogate keys from dimension tables:

- Lookup DimDate
- Lookup DimLocation
- Lookup DimRoadCondition
- Lookup DimCasualty

These lookups mapped source business attributes to their corresponding dimension surrogate keys.

4. Final Load

After all lookups and derived columns were completed, the transformed rows were inserted into the FactAccident table using an OLE DB Destination.

The fact table stores the measurable details of each accident transaction, such as vehicle count, severity score, and process-related timestamps.

Main components used

- OLE DB Source
- Derived Column
- Data Conversion
- Lookup transformations
- OLE DB Destination

To integrate all staged accident-related data and load the central fact table using resolved surrogate keys from the dimensions.

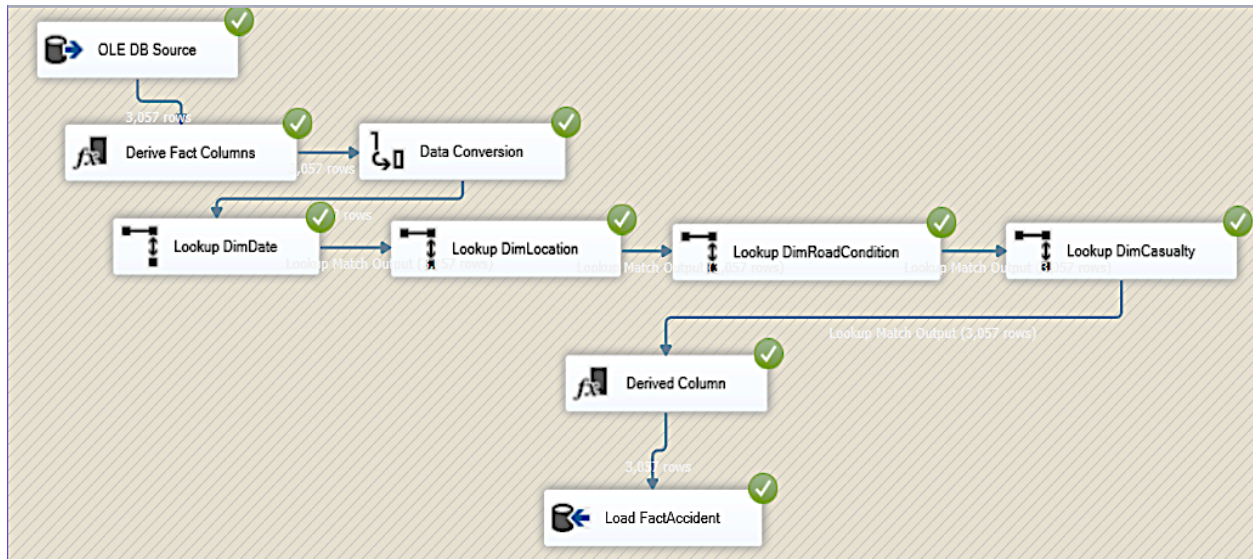


Figure 13: Data Flow Transform and Load Accident Fact

5.4.5 Validation of Package 2 Load

After the successful execution of UKAccident_Load_DW.dtsx, the contents of the data warehouse were validated using SQL queries. The validation confirmed that the dimensions and fact table were populated correctly.

The final validated record counts were as follows:

- DimDateCount = 365
- DimLocationCount = 2179
- DimRoadConditionCount = 60
- DimCasualtyCount = 1109
- FactAccidentCount = 3057

These counts confirm that:

- unique records were correctly loaded into dimension tables
- all source rows were successfully transformed and loaded into the fact table
- the ETL logic produced a consistent and complete warehouse structure

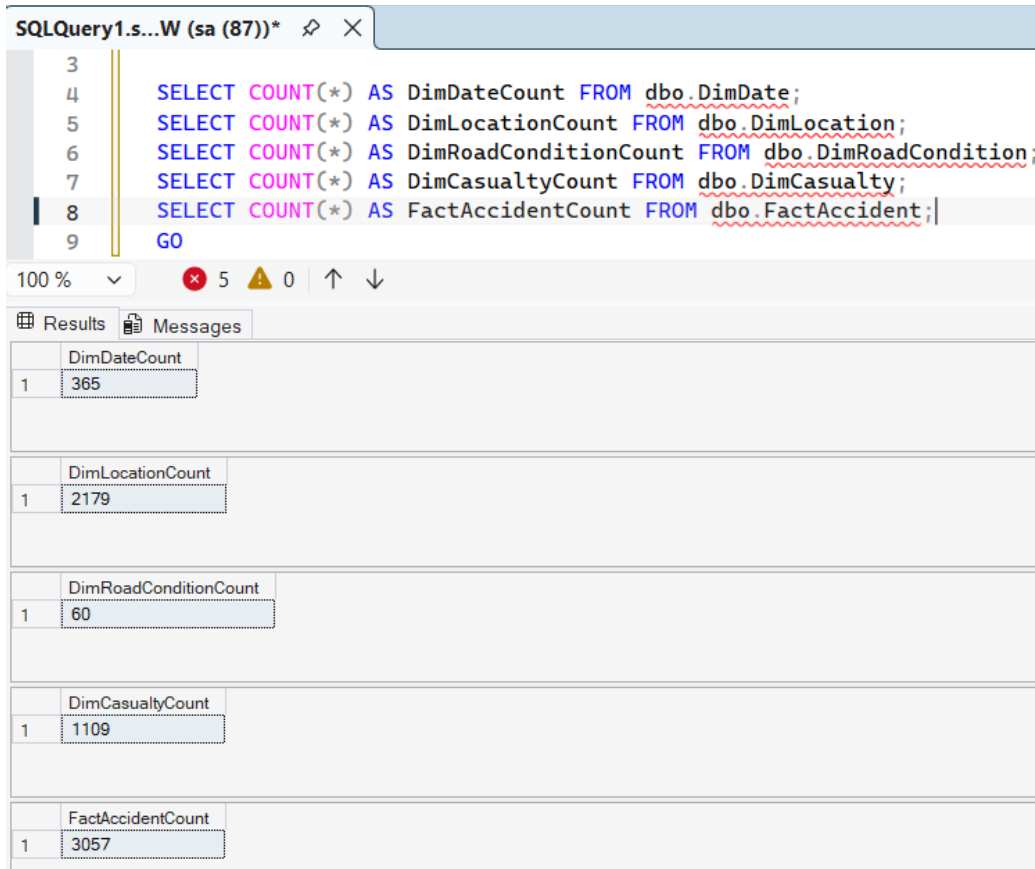


Figure 14: Validation results for dimension and fact tables after execution of UKAccident_Load_DW.dtsx

5.5 Error Handling and Data Quality Checks

During the ETL development, several data quality and execution issues were encountered and handled to ensure reliable data loading.

One of the main challenges was **data type mismatches**, especially between Unicode and non-Unicode strings. This issue was resolved using the **Data Conversion transformation** in SSIS to ensure compatibility between staging data and dimension tables.

Another issue was **duplicate records** when loading dimension tables. This was handled using:

- SELECT DISTINCT in source queries
- Sort transformation with duplicate removal

Additionally, **string truncation errors** occurred when importing CSV data due to insufficient column lengths. This was resolved by increasing column sizes in destination tables and aligning data types correctly.

To ensure consistency during multiple executions, dimension and fact tables were cleared before loading using SQL tasks. This prevented duplicate inserts when rerunning packages.

Overall, proper handling of data types, duplicates, and execution order ensured a stable ETL process.

5.6 ETL Execution Order

The correct execution order of ETL tasks is essential to maintain referential integrity in the data warehouse.

The execution order implemented in this project is as follows:

1. Load staging tables (Package 1)
2. Load dimension tables (Package 2)
 - DimLocation
 - DimRoadCondition
 - DimCasualty
 - DimDate
3. Load fact table (Package 2)
4. Update accumulating fact table (Package 3)

This order ensures that:

- Dimension tables are populated before fact table loading
- Lookup transformations can successfully resolve surrogate keys
- No foreign key violations occur

6 Step 6:Accumulating Fact Table

6.1 Overview

An accumulating fact table was implemented to track the lifecycle of accident transactions. Unlike regular fact tables, accumulating fact tables are updated over time as new information becomes available.

This allows analysis of how long it takes for a transaction to be completed.

6.2 Fact Table Extension

The FactAccident table was extended with the following additional attributes:

- accm_txn_create_time
- accm_txn_complete_time

- txn_process_time_hours

The accm_txn_create_time represents the time when the record is first created, while the accm_txn_complete_time represents when the process is completed.

6.3 Implementation Logic

During the initial ETL load:

- accm_txn_create_time was set using the system date
- accm_txn_complete_time was left NULL

A separate dataset containing completion times was prepared with:

- AccidentTxnID
- accm_txn_complete_time

A second ETL process was then used to update the fact table using this dataset.

The processing time was calculated using:

```
DATEDIFF(hour, accm_txn_create_time, accm_txn_complete_time)
```

This value was stored in txn_process_time_hours.

6.4 Package 3 - ETL Package for Fact Update

A separate SSIS package was created to update the accumulating fact table.

Steps involved:

1. Extract completion time dataset
2. Perform lookup using AccidentTxnID
3. Update matching records in FactAccident
4. Calculate processing time

The update was performed using an OLE DB Command transformation.

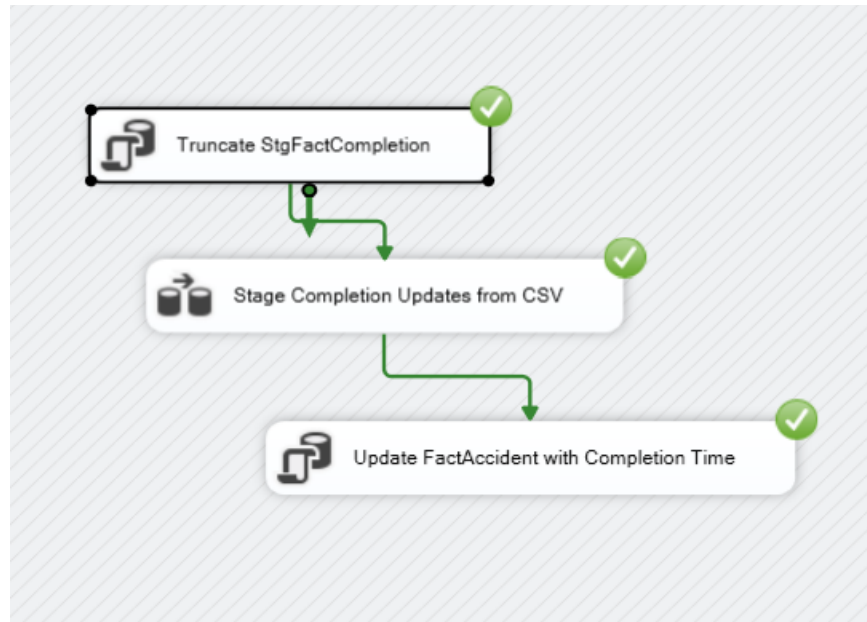


Figure 15: SSIS Package – Accumulating Fact Update Process

This figure shows the control flow of the SSIS package used to update the accumulating fact table with completion time and processing duration.

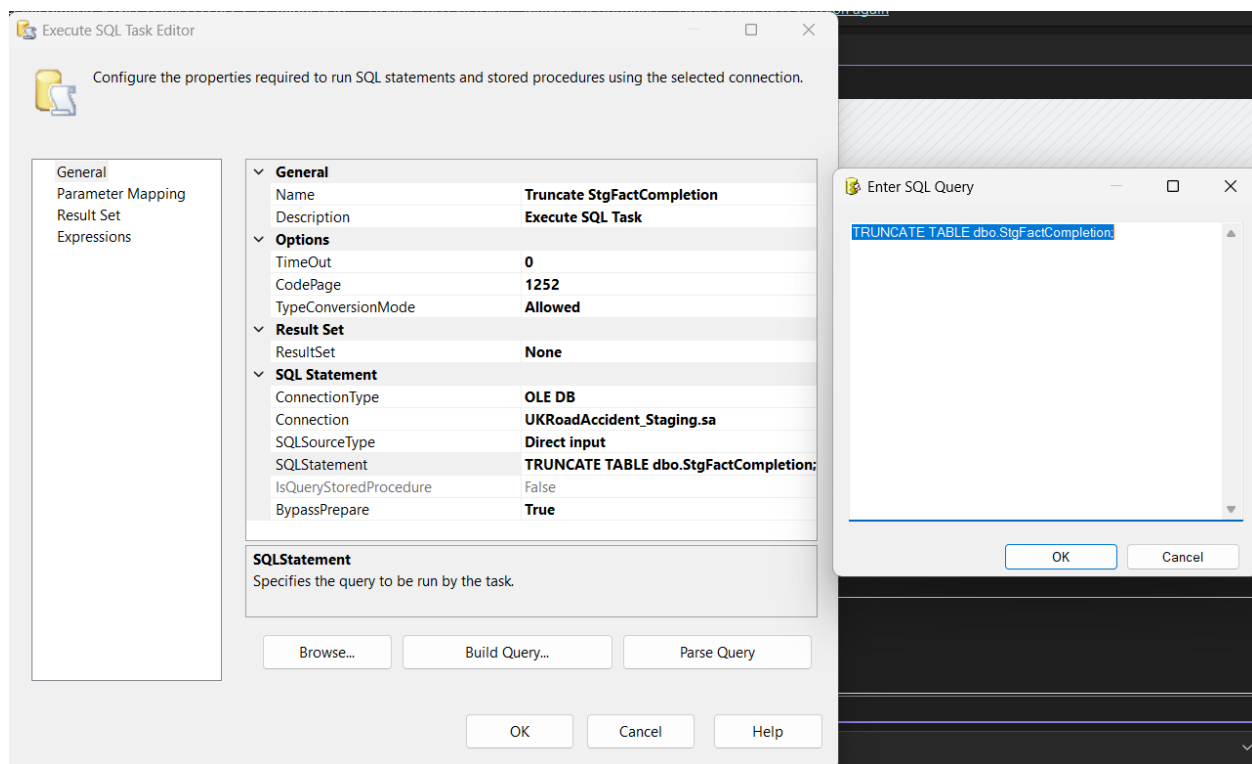


Figure 16 :Execute SQL Task – Truncate Staging Table

This task clears the staging table before loading new completion data, ensuring that no duplicate or outdated records remain from previous executions.

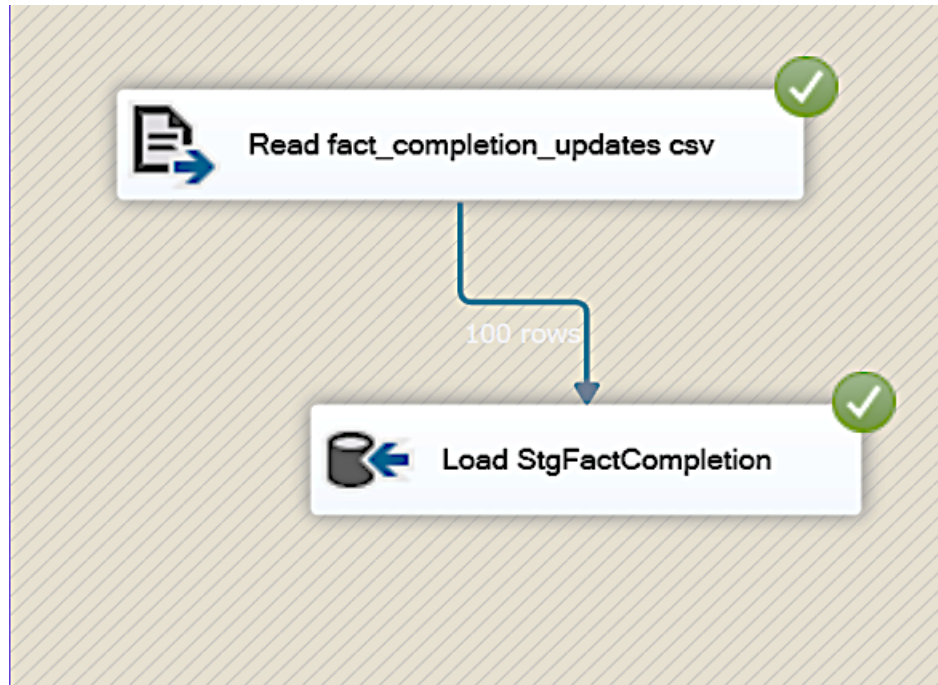


Figure 17: Data Flow – Load Completion Data into Staging

This data flow extracts completion time data from a CSV file and loads it into the staging table. The staging layer is used to temporarily store completion timestamps before updating the fact table.

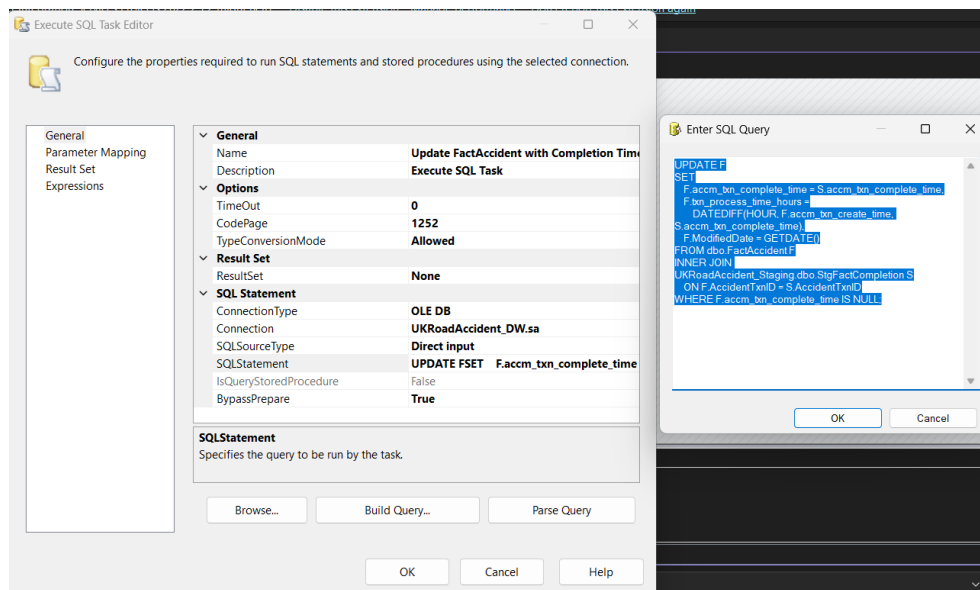


Figure 18 - Execute SQL Task (Update Fact)

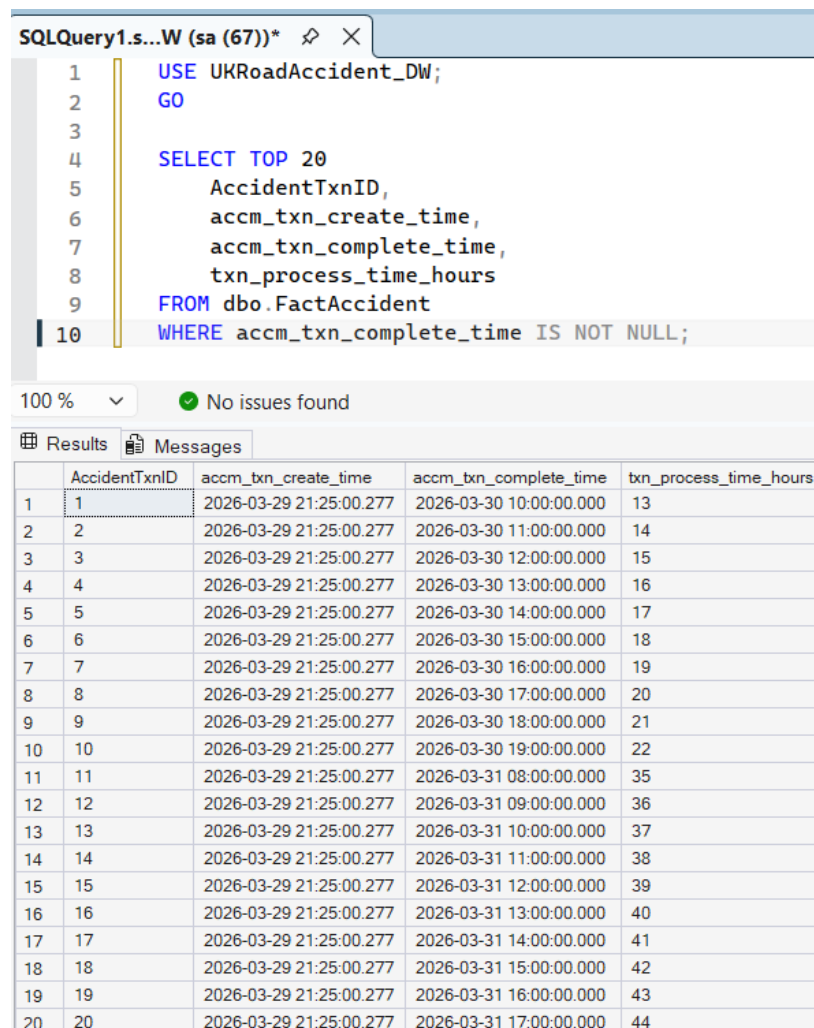
This task updates the FactAccident table by setting the completion time for each transaction and calculating the processing duration using the difference between creation and completion timestamps.

6.5 Validation

After executing the update process:

- Completion times were successfully updated
- Processing time values were correctly calculated
- No unmatched transaction IDs were found

The correctness was verified using SQL queries.



SQLQuery1.s...W (sa (67))*

```

1  USE UKRoadAccident_DW;
2  GO
3
4  SELECT TOP 20
5      AccidentTxnID,
6      accm_txn_create_time,
7      accm_txn_complete_time,
8      txn_process_time_hours
9  FROM dbo.FactAccident
10 WHERE accm_txn_complete_time IS NOT NULL;

```

100 % No issues found

	AccidentTxnID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	1	2026-03-29 21:25:00.277	2026-03-30 10:00:00.000	13
2	2	2026-03-29 21:25:00.277	2026-03-30 11:00:00.000	14
3	3	2026-03-29 21:25:00.277	2026-03-30 12:00:00.000	15
4	4	2026-03-29 21:25:00.277	2026-03-30 13:00:00.000	16
5	5	2026-03-29 21:25:00.277	2026-03-30 14:00:00.000	17
6	6	2026-03-29 21:25:00.277	2026-03-30 15:00:00.000	18
7	7	2026-03-29 21:25:00.277	2026-03-30 16:00:00.000	19
8	8	2026-03-29 21:25:00.277	2026-03-30 17:00:00.000	20
9	9	2026-03-29 21:25:00.277	2026-03-30 18:00:00.000	21
10	10	2026-03-29 21:25:00.277	2026-03-30 19:00:00.000	22
11	11	2026-03-29 21:25:00.277	2026-03-31 08:00:00.000	35
12	12	2026-03-29 21:25:00.277	2026-03-31 09:00:00.000	36
13	13	2026-03-29 21:25:00.277	2026-03-31 10:00:00.000	37
14	14	2026-03-29 21:25:00.277	2026-03-31 11:00:00.000	38
15	15	2026-03-29 21:25:00.277	2026-03-31 12:00:00.000	39
16	16	2026-03-29 21:25:00.277	2026-03-31 13:00:00.000	40
17	17	2026-03-29 21:25:00.277	2026-03-31 14:00:00.000	41
18	18	2026-03-29 21:25:00.277	2026-03-31 15:00:00.000	42
19	19	2026-03-29 21:25:00.277	2026-03-31 16:00:00.000	43
20	20	2026-03-29 21:25:00.277	2026-03-31 17:00:00.000	44

Figure 19: Results of Accumulating Fact Table Update

This figure shows the updated records in the FactAccident table after executing the accumulating fact update process. The completion time has been successfully populated, and the processing time in hours has been calculated based on the difference between creation and completion timestamps.

7 Step 7: Conclusion

This project successfully implemented a complete data warehousing solution for the UK road accident dataset.

The solution demonstrated:

- Integration of multiple heterogeneous data sources
- Implementation of a structured staging layer
- Design of a star schema data warehouse
- Development of ETL processes using SSIS
- Handling of real-world data quality issues
- Implementation of an accumulating fact table

The final system enables efficient analysis of accident data based on time, location, road conditions, and casualty attributes.

The design is scalable and can be extended further to support advanced analytics and reporting using tools such as SSAS and Power BI.

8 Assumptions

The following assumptions were made during the implementation:

- Each accident occurs at a single location
- Each accident has one set of road conditions
- Multiple casualty records may exist for a single accident
- Missing values are treated as NULL
- Age groups are derived based on predefined ranges